

Week 09 Weekly Exercises

Objectives

- Investigate FFI in rust
- Understand what unsoundness means
- Implement your own RC type with unsafe
- Short week of exercises, so more time for assignment

Activities To Be Completed

The following is a list of all the activities available to complete this week...

- **Rust2C**
- **Unsound Unsafe**
- **MyRc**

The following practice activities are optional and are not **marked, or required** to be completed for the week.

None - all exercises this week are marked.

Preparation

Before attempting the weekly exercises you should re-read the relevant lecture slides and their accompanying examples.

Getting Started

Create a new directory for this week's exercises called `lab09`, change to this directory, and fetch the provided code for this week by running these commands:

```
$ mkdir lab09
$ cd lab09
$ 6991 fetch lab 09
```

Or, if you're not working on CSE, you can download the provided code as a [tar file](#).

EXERCISE:

Rust2C

In this exercise, you are tasked with calling C code from within Rust!

You have been provided `secretc.h`, a header file which contains the definition of a secret C function:

```
int secret_c_calculation(int x, double y, char z);
```

This is joined by `libsecretc.a`, the compiled C code ready to be linked through a Foreign Function Interface (FFI) with the Rust code!

You do not need to understand the C source code, and in-fact, we do not *provide* you the C source code. You are only required to *call* the C code from within Rust, and make use of the result it provides.

You will need to write both the C function definition (using the header file as a guideline) and call that C function from within main yourself.

The linking should already be taken care of for you (with a `build.rs` script and a `#[link(...)]` attribute), so if your code is written correctly, you should be able to simply use `6991 cargo` as usual.

Note that since this exercise makes use of a pre-compiled C object file, we can only guarantee this exercise building correctly on x86_64-unknown-linux-gnu systems. This includes most Linux systems (including CSE) and most WSL installs, but not Windows itself nor MacOS. If you aren't running on a typical Linux system, you should work on this exercise on CSE (e.g. VLAB, SSH, lab machines, etc.). Although FFI generally works on many platforms, it is the C library in this case that restricts us to this target.

For example,

```
$ 6991 cargo run -- 42 123.456 F
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/rust2c 42 123.456 F`
The secret value is 176
```

When you think your program is working, you can use autotest to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running give:

```
$ 6991 give--crate
```

You must run give before **Week 10 Sunday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with give must be entirely your own.

SOLUTION:

main.rs

```
use std::{fmt::Debug, str::FromStr};

use libc::{c_int, c_double, c_char};

const USAGE_STR: &str = "Usage: cargo run -- <x> <y> <z>";

#[link(name = "secretc")]
extern "C" {
    fn secret_c_calculation(x: c_int, y: c_double, z: c_char) -> c_int;
}

fn next_arg<T: FromStr>(args: &mut impl Iterator<Item = String>, message: &str) -> T
where
    <T as FromStr>::Err: Debug,
{
    args.next().expect(USAGE_STR).parse().expect(message)
}

fn main() {
    let mut args = std::env::args().skip(1);

    let x: i32 = next_arg(&mut args, "x must be i32");
    let y: f64 = next_arg(&mut args, "y must be f64");
    let z: char = next_arg(&mut args, "z must be char");

    let secret_value = unsafe { secret_c_calculation(x, y, z as i8) };

    println!("The secret value is {secret_value}");
}
```

EXERCISE:

Unsound Unsafe

In this exercise, you have been provided the `unsound_unsafe` crate.

This crate attempts to (very dangerously) implement a singly-linked list data structure making liberal use of raw pointers instead of taking advantage of Rust's safety guarantees provided through its usual types (e.g. references).

Nevertheless, we are making use of this linked list, and it seems to generally work okay! We initialise a new linked list comprised of numbers given as command-line arguments, and perform the following operations:

- Print the list;
- Delete the first element in the list;
- Print the list again;
- Free the list.

Running the program with the list 1 2 3 can be observed as follows:

```
$ 6991 cargo run -- 1 2 3
    Finished dev [unoptimized + debuginfo] target(s) in 0.01s
    Running `target/debug/unsound_unsafe 1 2 3`
=== PRINTING LIST ===
1
2
3
=====
=== PRINTING LIST ===
2
3
=====
```

It seems that most of these operations work correctly (i.e. they are sound), however there appears to be a soundness issue when the `list_delete_first` function is called in the case of an already empty list! Users of our dangerous linked-list structure have reported segmentation faults in this specific scenario!

```
$ 6991 cargo run --
    Finished dev [unoptimized + debuginfo] target(s) in 0.01s
    Running `target/debug/unsound_unsafe`
=== PRINTING LIST ===
=====
[1]      1402912 segmentation fault  6991 cargo run --
```

Your task is first to locate the unsoundness using [Miri](#). Miri is already installed on CSE systems, so you can try running your code like such:

```
$ 6991 cargo +nightly miri run -- 1 2 3
Preparing a sysroot for Miri (target: x86_64-unknown-linux-gnu)... done
    Finished dev [unoptimized + debuginfo] target(s) in 0.01s
    Running `/home/cs6991/.rustup/toolchains/nightly-x86_64-unknown-linux-gnu/bin/cargo-miri runner
target/miri/x86_64-unknown-linux-gnu/debug/foo 1 2 3`
=== PRINTING LIST ===
1
2
3
=====
=== PRINTING LIST ===
2
3
=====
```

Once you understand how to use Miri, try replicating the input demonstrating a segmentation fault from above in order to track down where this issue arises.

Finally, it's time to fix the unsoundness! You are **only** permitted to modify the `list_delete_first` function. You may not modify any other code.

Once you are satisfied that your code is most-likely sound, you should run it once again through Miri! Furthermore, the autotests for this exercise will test your code using Miri on a few different cases.

When you think your program is working, you can use autotest to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running `give`:

```
$ 6991 give-crate
```

You must run `give` before **Week 10 Sunday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with `give` must be entirely your own.

SOLUTION:

main.rs

```

use std::ptr::null_mut as null;

/// Delete the first node in a linked list.
/// This function will return the new list.
///
/// If the list is empty (i.e. `list == null()`),
/// then this function should return `null()`.
///
/// # Safety
///
/// The list must be a valid linked list,
/// as is defined on the type `List`.
///
/// The list may have zero or more elements.
///
unsafe fn list_delete_first(list: *mut List<i32>) -> *mut List<i32> {
    if list == null() { return null(); }

    // //! SAFETY: The list must be a valid linked list,
    //             so we get a pointer to the second node
    //             for later!
    //             TODO: hmm... is this definitely sound?
    let next = unsafe { (*list).next };

    // //! SAFETY: The list must be a valid linked list,
    //             so we can free the first node.
    //             TODO: hmm... is this definitely sound?
    //             ... the first node exists... right?
    unsafe { free_node(list); }

    return next;
}

////////////////////////////////////
// DO NOT MODIFY CODE BELOW THIS POINT //
////////////////////////////////////

fn main() {
    let list = my_linked_list();

    // //! SAFETY: my_linked_list() always returns a valid
    //             linked list, which fulfils the safety
    //             preconditions of all other functions!
    unsafe {
        print_list(list);
        let list = list_delete_first(list);
        print_list(list);

        free_list(list);
    }
}

/// A linked list node.
///
/// Note that any individual node may also
/// represent a list in its entirety, due to
/// the nature of how linked lists work.
///
/// # Validity
///
/// A linked list (of type *mut List<T>)
/// is considered valid if it is either:
///
/// 1. Null (i.e. list == std::ptr::null_mut())
///
/// 2. A heap allocated value
///    (constructed from a `Box`),
///    whose `next` field is
///    *also* a valid linked list.
struct List<T> {
    value: T,
    next: *mut List<T>,
}

```

```

fn my_linked_list() -> *mut List<i32> {
    let nums = std::env::args()
        .skip(1)
        .map(|arg| arg.parse::<i32>()).expect(&format!("Failed to parse {arg} as i32")))
        .collect::<Vec<_>>();

    let mut curr_node = None;
    for value in nums.into_iter().rev() {
        curr_node = Some(Box::into_raw(Box::new(
            List {
                value,
                next: curr_node.unwrap_or_else(null),
            }
        )));
    }

    curr_node.unwrap_or_else(null)
}

/// Print a linked list
///
/// # Safety
///
/// The list must be a valid linked list,
/// as is defined on the type `List`.
///
/// The list may have zero or more elements.
unsafe fn print_list(list: *mut List<i32>) {
    println!("=== PRINTING LIST ===");
    let mut curr = list;
    while curr != std::ptr::null_mut() {
        // //! SAFETY: `curr` is not `null`,
        //             therefore it is safe to dereference
        //             and access its fields.
        println!("{}", unsafe { (*curr).value });

        // //! SAFETY: ... as above
        curr = unsafe { (*curr).next };
    }
    println!("=====");
}

/// Free a linked list node
///
/// # Safety
///
/// The node must be a heap allocated value,
/// originally constructed from a `Box`.
/// Specifically, it must also *not* be `null`.
///
/// The node must currently be valid to *own*.
///
/// There are no requirements on `next`.
unsafe fn free_node(node: *mut List<i32>) {
    // //! SAFETY: The node is a heap allocated value
    //             constructed from a `Box`, that is
    //             currently candidate to own.
    //
    //             Therefore, `Box::from_raw` is sound.
    drop(unsafe { Box::from_raw(node) });
}

/// Free a linked list
///
/// # Safety
///
/// The list must be a valid linked list,
/// as is defined on the type `List`.
///
/// The list may have zero or more elements.
unsafe fn free_list(list: *mut List<i32>) {
    let mut curr = list;
    while curr != std::ptr::null_mut() {
        let to_free = curr;

```

```
// //! SAFETY: `curr` is not `null`,
//           therefore it is safe to dereference
//           and access its fields.
curr = unsafe { (*curr).next };

// //! SAFETY: `to_free` came from `curr`,
//           which was a non-null linked list
//           node. Due to the requirements on
//           `List`, `to_free` must be sound
//           to free with `free_node`.
unsafe { free_node(to_free) };
}
}
```

EXERCISE:

MyRc

In this exercise, we will be implementing our own simple Rc type.

Your task is to build a datastructure, called MyRc which has the following properties:

- The type should allow multiple people to own a value.
- The value should live until the last MyRc is dropped.
- The value should be dropped when the last MyRc is dropped.

There are hints for each function you should complete, in the starter code.

No changes to main.rs should be required, only lib.rs

You will need to write unsafe code in this activity - don't forget SAFETY comments! E.G [the rust API guidelines](#)

HINT:

The documentation for the [Box::from_raw](#) method may be useful.

The documentation for the [Box::into_raw](#) method may be useful.

When you think your program is working, you can use autotest to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running give:

```
$ 6991 give-crate
```

You must run give before **Week 10 Sunday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with give must be entirely your own.

SOLUTION:

lib.rs


```

use std::ops::Deref;

struct Inner<T> {
    refcount: usize,
    data: T
}

pub struct MyRc<T> {
    inner: *mut Inner<T>
}

impl<T> MyRc<T> {
    pub fn new(value: T) -> Self {
        MyRc {
            inner: Box::into_raw(Box::new(Inner {
                refcount: 1,
                data: value
            }))
        }
    }
}

impl<T> Clone for MyRc<T> {
    fn clone(&self) -> Self {
        unsafe {
            (*self.inner).refcount += 1;
        }
        MyRc {
            inner: self.inner
        }
    }
}

impl<T> Drop for MyRc<T> {
    fn drop(&mut self) {
        unsafe {
            (*self.inner).refcount -= 1;

            if (*self.inner).refcount == 0 {
                drop(Box::from_raw(self.inner));
            }
        }
    }
}

impl<T> Deref for MyRc<T> {
    type Target = T;

    fn deref(&self) -> &T {
        unsafe {
            &(*self.inner).data
        }
    }
}

```

main.rs


```

use myrc_lib::MyRc;

struct Owner {
    name: String,
}

struct Gadget {
    id: i32,
    owner: MyRc<Owner>,
}

// Example used from The Book: https://doc.rust-lang.org/std/rc/index.html#examples
fn main() {
    // Create a reference-counted `Owner`.
    let gadget_owner: MyRc<Owner> = MyRc::new(
        Owner {
            name: "Gadget Man".to_string(),
        }
    );

    // Create `Gadget`s belonging to `gadget_owner`. Cloning the `Rc<Owner>`
    // gives us a new pointer to the same `Owner` allocation, incrementing
    // the reference count in the process.
    let gadget1 = Gadget {
        id: 1,
        owner: MyRc::clone(&gadget_owner),
    };
    let gadget2 = Gadget {
        id: 2,
        owner: MyRc::clone(&gadget_owner),
    };

    // Dispose of our local variable `gadget_owner`.
    drop(gadget_owner);

    // Despite dropping `gadget_owner`, we're still able to print out the name
    // of the `Owner` of the `Gadget`s. This is because we've only dropped a
    // single `Rc<Owner>`, not the `Owner` it points to. As long as there are
    // other `Rc<Owner>` pointing at the same `Owner` allocation, it will remain
    // live. The field projection `gadget1.owner.name` works because
    // `Rc<Owner>` automatically dereferences to `Owner`.
    println!("Gadget {} owned by {}", gadget1.id, gadget1.owner.name);
    println!("Gadget {} owned by {}", gadget2.id, gadget2.owner.name);

    // At the end of the function, `gadget1` and `gadget2` are destroyed, and
    // with them the last counted references to our `Owner`. Gadget Man now
    // gets destroyed as well.
}

```

Submission

When you are finished each exercise make sure you submit your work by running give.

You can run give multiple times. Only your last submission will be marked.

Don't submit any exercises you haven't attempted.

If you are working at home, you may find it more convenient to upload your work via [give's web interface](#).

Remember you have until **Week 10 Sunday 21:00:00** to submit your work.

You cannot obtain marks by e-mailing your code to tutors or lecturers.

Automarking will be run several days after the submission deadline, using test cases different to those autotest runs for you. (Hint: do your own testing as well as running autotest.)

After automarking is run you can [view your results here](#).

Weekly exercise Marks

When all exercises of a week are automarked you should be able to view the the marks [via give's web interface](#) or by running this command on a CSE machine:

COMP6991 23T1: Solving Modern Programming Problems with Rust is brought to you by
the [School of Computer Science and Engineering](#)
at the [University of New South Wales](#), Sydney.

For all enquiries, please email the class account at cs6991@cse.unsw.edu.au

CRICOS Provider 00098G

[Login as tutor](#)